

SmartBus AI: A Mobile-First Real-Time Bus Tracking and Crowd-Aware Journey Planning System for the BMTC Network

Manjunath Charvik N, Shiva Kumar B, Mohsin Abdul Bari, P Puneetraj School of Computer Science and Engineering REVA University, Bengaluru, India {manjunathcharvik, shivakumarb, mohsinabdulbari, ppuneetraj}@gmail.com

Project Guide: Prof. Suvarna Hugar Assistant Professor, School of Computer Science and Engineering REVA University, Bengaluru, India suvarna.hugar@reva.edu.in

ABSTRACT

City buses in Bengaluru carry close to four million people every day, but the rider experience has not kept pace with the size of the network. The Bengaluru Metropolitan Transport Corporation (BMTC) operates 4,203 routes across 6,006 stops, and yet a typical commuter still waits at the bus stop with no idea whether a bus is on its way, how full it will be when it arrives, or whether a better route exists for the same trip. This work describes SmartBus AI, a cross-platform mobile application that pulls these missing pieces into one place. The system is built on the official BMTC General Transit Feed Specification (GTFS) feed, augmented by an in-process simulator that produces a steady stream of 240 live vehicles, and it surfaces the data through a small typed REST API consumed by an Expo / React Native client. Three things make the application different from a thin GTFS viewer. First, every live bus carries an AI-derived occupancy band that is exposed to the rider as a plain-language label such as "*Seats available*" or "*Very crowded*" rather than as a colour alone. Second, the journey planner ranks its top three results with descriptive tags (Recommended, Fastest, Comfortable) so that a first-time rider can pick a bus without parsing a table. Third, the entire interface is tuned for elderly accessibility, with a 48 pt minimum touch target, large display typography, and indicators that pair colour with icon and text. Functional testing on commodity hardware shows median API latency of 2 ms for the live-bus endpoint and 35 ms for the route directory of all 4,203 routes, with an end-to-end first-meaningful-content time below two seconds on the mobile client. The architecture is open: the simulator can be replaced with a public GTFS-Realtime feed when one becomes available, and the heuristic crowd classifier can be swapped for a learned model without changing the API surface.

INDEX TERMS Intelligent transportation systems, GTFS, real-time public transit, bus

crowd prediction, mobile UX, accessibility, BMTC, Bengaluru, React Native, Expo.

I. INTRODUCTION

Public transport in Indian metros sits at an awkward inflection point. The buses are there, the routes exist, and the agencies have begun publishing schedule data in standard formats, but the everyday rider still leans heavily on word of mouth and visual confirmation at the stop. In Bengaluru, BMTC operates more than six thousand vehicles across over four thousand routes that together serve several million daily passengers [1], [19]. The volume is enormous. The information layer on top of it, however, is thin.

Three recurring frustrations show up in any informal survey of BMTC riders. The first is uncertainty about whether a particular route is currently in service, especially during off-peak hours, late evenings, and weekends. Static schedules do not capture cancellations or delays, and there is no public real-time feed yet from the agency. The second is the inability to anticipate in-vehicle crowding before boarding. For an elderly rider, a parent with a small child, or a woman travelling at night, knowing that the next bus will be standing-room-only changes the decision from "board this one" to "wait for the next one". The third is the difficulty of discovering a good route between two stops when the catalogue is in the thousands and many routes overlap. New riders, occasional travellers, and visitors to the city are particularly affected by this discovery problem.

Most of the existing third-party transit apps tackle one of these problems at a time. Some show static schedules. Some show partial live tracking on a small set of routes. Very few combine the three things a rider actually needs in a single screen, and almost none are designed with the elderly first-time user in mind. The interfaces tend to be small-text, dense, and colour-coded in ways that fall apart for a rider

with mild presbyopia or any form of colour-vision deficiency.

The work reported in this paper is an attempt to close that gap. We have built a unified mobile application, called SmartBus AI, that brings live tracking, AI-assisted crowd prediction, journey planning, and route discovery together behind a single calm interface. The application is implemented in Expo and React Native, talks to a typed REST API written in TypeScript on Node.js, and consumes the official BMTC GTFS feed of 4,203 routes and 6,006 stops. Live vehicle positions are produced by a deterministic simulator on a 12-second tick because BMTC has not yet released a public GTFS-Realtime feed, but the architecture is designed so that swapping the simulator for a real feed is a single-file change. Crowd predictions are produced by a heuristic feature-weighted classifier whose output is rendered to the rider as an icon plus a colour plus a plain-language label, never as a colour by itself. The whole interface uses a premium dark theme tuned for OLED displays, big display-grade typography, and a strict 48 pt minimum touch target on every interactive control. The result is an application that looks like a contemporary mobility product but reads like a large-print rider's guide.

The contributions of this work can be summarised as follows.

1. An end-to-end open architecture that ingests the official BMTC GTFS dataset, augments it with a deterministic vehicle simulator, and serves the result through a small typed REST API consumable by mobile and web clients alike.
2. A heuristic AI-assisted crowd-prediction layer that classifies every live bus into one of three semantically meaningful occupancy bands using time-of-day, route-popularity, vehicle-type, and day-of-week features. The output is exposed both numerically and as a friendly label.
3. A custom SVG-based route mini-map that renders the polyline geometry of any route together with its stops and the live bus position, with no dependency on Google Maps, Mapbox, or any other paid tile service.
4. A premium dark design system whose typography, contrast, and tap-target sizing are explicitly calibrated against the WCAG 2.2 success criteria and the Apple Human Interface Guidelines for elderly accessibility, while still preserving the visual polish of contemporary first-tier mobility apps.

5. A reference implementation that runs on a single commodity Node.js process with sub-50 ms median latency on every endpoint other than the exhaustive search call, demonstrating that an AI-augmented BMTC tracking platform is feasible without the operational overhead of a large agency back-end.

The remainder of the paper is organised as follows. Section II surveys the related work in real-time public transit, crowd prediction, and accessible mobile design. Section III describes the BMTC GTFS dataset and the data-augmentation pipeline. Section IV presents the system architecture. Section V documents the AI crowd-prediction methodology. Section VI explains the user-experience design rationale and shows the resulting screens. Section VII reports the evaluation results. Section VIII discusses the limitations of the current build and outlines the future work. Section IX concludes.

II. RELATED WORK

A. Real-Time Public Transit Information Systems

The General Transit Feed Specification (GTFS), originally created by Google in collaboration with TriMet of Portland in 2006 [2], has become the de-facto standard for static transit data. Its companion specification, GTFS-Realtime [3], extends the data model with vehicle positions, trip updates, and service alerts. Most large transit agencies in North America and Europe now publish both feeds, and a healthy ecosystem of consumer applications such as Citymapper, Transit, and Moovit has grown up around them. In India, the picture is more uneven. BMTC publishes a static GTFS feed that this project consumes, but it does not yet expose a public real-time feed. This is a recurring theme across Indian metropolitan agencies, and it is the main reason the present work uses an in-process simulator for live vehicle state.

Catalá-Prat and colleagues [4] showed that arrival predictions in mid-sized European cities can be made acceptably accurate even when Automatic Vehicle Location (AVL) coverage is sparse, by combining the static GTFS schedule with whatever real-time pings are available. The hybrid model outperformed both the schedule-only baseline and the AVL-only baseline. Their approach is encouraging for BMTC because even partial future AVL deployment, combined with the GTFS feed already in use here, would yield meaningful prediction quality. Wessel and Farber [5] further demonstrated that static GTFS feeds,

when paired with intelligent client-side rendering, can support a wide range of consumer applications without any real-time infrastructure at all.

B. Bus Crowd and Occupancy Prediction

Predicting in-vehicle occupancy has been studied with several sensing modalities. Wang and colleagues [6] used data from Automatic Passenger Counters (APC), a hardware sensor mounted at the bus door, together with a hybrid deep learning model to forecast passenger flow. Their results were strong but the approach assumes APC instrumentation that BMTC's older fleet largely lacks. Liu and colleagues [7] explored vision-based passenger counting using deep convolutional networks on the onboard CCTV feed. Again, the model is good, but the sensor footprint is too heavy for retrofit deployment on a legacy bus fleet. Pinelli and colleagues [8] proposed a lighter alternative based on Wi-Fi probe-request counting from passengers' phones near the bus, but this approach raises real privacy concerns and has not been widely adopted.

When ground-truth occupancy labels are sparse, Han and colleagues [9] showed that a heuristic feature-weighted classifier using time-of-day, route-popularity, and vehicle-type features yields acceptable performance for a coarse three-class occupancy band. This is the regime SmartBus AI operates in today, and the heuristic crowd model described in Section V is directly inspired by their findings. The architecture of the present system is, however, designed to allow an APC-trained learned model to be dropped in later without changing the API.

C. Journey Planning at City Scale

Multimodal journey planning across a large transit network is a classical problem with mature solutions. The RAPTOR algorithm of Bast and colleagues [10] and its many derivatives underpin most production-grade transit planners. RAPTOR scales to country-wide networks and handles complex transfer patterns elegantly. For a single-mode bus network like BMTC, however, riders mostly need direct routes between two named stops, and a much simpler exhaustive scoring scan over the route corpus remains practical when paired with appropriate caching. The simpler approach has the advantage of being trivially explainable to the rider, which matters for accessibility.

D. Mobile UX for Elderly Transit Users

Several studies of older adults' use of smartphone transit apps converge on the same three barriers:

small text, low colour contrast, and ambiguous icon-only controls. Kurniawan [11] reported these findings across a multi-method investigation of older mobile-phone users, and Harte and colleagues [12] confirmed them in a human-centred design study of connected health systems, which face very similar interface constraints. The Web Content Accessibility Guidelines (WCAG) 2.2 [13] codify minimum contrast ratios and recommend a 24×24 CSS pixel minimum touch target. Apple's Human Interface Guidelines [14] go further and recommend 44×44 pt. Material Design [15] recommends 48 dp, which is the most generous of the three and the one this work follows on every interactive control.

E. The React Native and Expo Stack

The React Native runtime [16] and the Expo development platform [17] have substantially reduced the cost of building production-grade cross-platform mobile applications. Combined with a server-state cache such as TanStack Query [18] and a typed JavaScript framework on the back end, a small team can now ship a real-time transit experience without the engineering overhead that previously confined such systems to large agencies. The classical usability heuristics of Nielsen [20] continue to provide a robust evaluation lens for the resulting interfaces.

F. Summary of the Gap

Prior work has made strong progress on each piece of the puzzle separately. Arrival prediction, crowd estimation, large-network journey planning, and accessible mobile design have all been studied carefully. What is much harder to find in the literature is a single deployed, commuter-facing product that puts these pieces together for an Indian metropolitan agency. SmartBus AI is an attempt to fill that gap.

III. DATASET

A. The BMTC GTFS Feed

The official BMTC GTFS feed is the foundation of the system. After cold-start ingestion, it gives us the entities listed in Table 1.

TABLE 1. BMTC GTFS entities used by SmartBus AI.

Entity	Approx. count	Purpose
Routes	4,203	Logical bus routes including Ordinary, Vajra (AC), Volvo, Airport, Metro Feeder, and Night Owl variants
Stops	6,006	Geo-located bus stops across the Bengaluru Metropolitan Region
Stop times	≈ 1.1 million	Scheduled arrival times by trip and stop
Shapes	≈ 12,000	Polyline geometries for each route variant
Calendar entries	1,800+	Weekday vs weekend service patterns

On ingestion, routes are normalised into six bus-type categories. Each category is mapped to a distinct colour and gradient that is then used consistently across the application. The mapping is shown in Table 2.

TABLE 2. Bus type categories and their UI colours.

Bus type	Hex colour	Typical use
Ordinary	#DC2626	Standard non-AC service
Vajra (AC)	#2563EB	Premium air-conditioned service
Volvo	#16A34A	Long-distance AC service
Airport	#DB2777	Vayu Vajra airport shuttles
Metro Feeder	#0891B2	Last-mile metro connections
Night Owl	#7C3AED	Late-night services

B. The Live-Bus Simulator

In the absence of a public GTFS-Realtime feed, the system ships with a deterministic in-process simulator. At server startup, up to `MAX_LIVE_ROUTES = 80` routes are sampled with weighting proportional to stop count, so that high-density corridors are over-represented in the live fleet. For each sampled route, `BUSES_PER_ROUTE = 3` virtual vehicles are instantiated. The result is a steady-state fleet of 240 buses.

Every twelve seconds the simulator advances each bus one tick along its route's stop sequence. The per-tick speed is sampled from a distribution that depends on the route type, so that a Volvo express bus advances faster on average than an Ordinary stop-everywhere bus. Each bus exposes the fields shown in Table 3.

TABLE 3. Per-bus state exposed by the simulator.

Field	Description
<code>routeId</code>	Internal route identifier
<code>routeNumber</code>	Public-facing route label, e.g. "244-C"
<code>routeName</code>	Origin → destination string
<code>currentStop</code>	Most recently passed stop
<code>nextStop</code>	Stop the bus is currently moving toward
<code>stopsCovered</code>	Index in the stop sequence
<code>totalStops</code>	Length of the stop sequence
<code>speed</code>	Instantaneous speed in km/h
<code>busType</code>	One of the six categories from Table 2
<code>crowdLevel</code>	Output of the heuristic classifier of Section V

The 12-second tick was chosen to match the polling interval of the mobile client, so that the system reaches a natural steady state under the expected load of one logged-in client per simulated bus.

IV. SYSTEM ARCHITECTURE

The project is organised as a pnpm monorepo containing four artifacts that share TypeScript types via project references. The artifacts are:

1. **api-server** — a Node.js / TypeScript REST service.
2. **smartbus-mobile** — the Expo / React Native application targeted at iOS and Android.
3. **smartbus** — a React + Vite web client mirroring the mobile experience for desktop preview.
4. **mockup-sandbox** — a component preview environment used during design iteration.

Sharing types across the monorepo means that any change to a back-end response shape produces a compile-time error in every client that consumes that shape. This caught several drift bugs during development.

A. Back-End API

The back-end exposes the endpoints listed in Table 4.

TABLE 4. REST endpoints exposed by the API server.

Method	Endpoint	Purpose
GET	/api/buses/live	Snapshot of all simulated live buses
GET	/api/routes	Directory of all 4,203 routes
GET	/api/routes/:id	Detailed route metadata, stop sequence, polyline shape
GET	/api/routes/:id/frequency?dayType=weekday weekend	Hourly frequency distribution
GET	/api/stops	Directory of all 6,006 stops
GET	/api/stops/:id/eta	Predicted arrivals at a stop
GET	/api/stops/:id/crowd	Crowd forecast for the next arrivals
GET	/api/search?source=&destination=	Source-destination journey planner

GTFS files are parsed once at cold start into in-memory indices keyed by route ID, by stop ID, and by a normalised stop-name trigram for fuzzy search. The trigram index is the workhorse of the search endpoint.

B. Mobile Client

The mobile client is built with Expo (React Native) using TypeScript throughout. File-based navigation comes from Expo Router, server-state caching from TanStack Query, gesture and entrance animations from Reanimated 4, and SVG rendering from `react-native-svg`. The application has three top-level tabs — **Live**, **Search**, and **Routes** — and two stacked detail screens for individual routes and stops.

A floating, blur-backed tab bar with both icons and text labels provides the global navigation. The active tab carries a soft glow indicator and the tap triggers a light haptic, which together give the rider an unambiguous feedback loop without being noisy.

Each screen subscribes to a small number of typed endpoints via TanStack Query. Refetch intervals are tuned to the real-time semantics of the underlying data. The live-bus snapshot is refetched every 12 seconds. Stop ETAs are refetched every 30 seconds. Static directories are not refetched automatically once loaded.

C. The Custom SVG Mini-Map

The decision not to use a paid mapping API was deliberate. Google Maps and Mapbox are excellent products but introduce a per-tile cost that scales with daily active users, an external dependency that the development team cannot patch, and a privacy footprint that is hard to audit. For the kind of route-shape preview the application actually needs — a polyline, the stops along it, and a single live bus marker — none of that complexity is justified.

The `RouteMiniMap` component is implemented entirely in `react-native-svg`. It accepts an array of `[longitude, latitude]` pairs together with a list of stop coordinates, defensively filters out invalid (NaN, null, or out-of-range) points, computes the bounding box of what remains, and projects the coordinates linearly into the SVG viewport. The polyline is rendered with the route-type gradient. Stop markers are drawn as small filled circles. Origin and destination labels are rendered as separate `Text` siblings rather than being nested inside one another, which respects the React Native rule that prohibits placing a `View` inside a `Text`. The live bus marker is drawn as a glowing filled circle on top of the polyline at the projected position.

The component renders identically on iOS, Android, and the web build.

V. AI CROWD-PREDICTION METHODOLOGY

The crowd-prediction module is the AI layer of the system. Its job is to map each live bus into one of three semantically meaningful occupancy bands. Because we do not have onboard passenger counts to learn from, the present implementation is a heuristic classifier whose feature weights were tuned by hand against rider expectations. The architecture is built so that a learned model can be dropped in later as soon as labelled data becomes available.

A. The Scoring Function

For a given bus, the crowd score is computed as

```

score(bus) = w_t · time_factor(t)
            + w_r · route_popularity(route)
            + w_b · bus_type_factor(type)
            + w_d · day_factor(day_of_week)
            + ε

```

The terms are interpreted as follows.

`time_factor(t)` is a triangular function that peaks during the morning commute window of 07:30–10:30 and the evening commute window of 17:30–20:30, and falls off at midday and late at night. The peaks were positioned by inspection of the BMTC schedule density.

`route_popularity(route)` is a normalised score in $[0, 1]$ that combines the route's stop count and historical service frequency. The intuition is that routes with many stops on busy corridors and with high service frequency tend to carry more passengers per vehicle, simply because more passengers are waiting to board.

`bus_type_factor(type)` adjusts for the typical capacity and ridership profile of each bus class. Volvo and Vajra services tend to be denser at peak hours because of the white-collar commuter profile. Ordinary buses dominate the off-peak and weekend traffic.

`day_factor(day_of_week)` reduces the score on Sundays and on public holidays.

Finally, ϵ is a small zero-mean noise term that prevents the score from being identical across two buses on the same route at the same minute. Without it, the displayed crowd indicators would feel mechanical.

The score is split into three bands by two fixed thresholds: $\tau_{\text{low}} = 0.33$ and $\tau_{\text{high}} = 0.67$. Bands below τ_{low} are labelled Low. Bands between the thresholds are Medium. Bands above τ_{high} are High.

B. Rendering the Output to the Rider

The classified band is exposed by the API as a small enum, but the way it is rendered to the rider is what makes the feature accessible. A dedicated `CrowdRow` component combines, for every band, three independent perceptual channels: an icon, a colour, and a plain-language label. The combinations are listed in Table 5.

TABLE 5. Perceptual mapping of crowd bands.

Band	Colour	Icon	Label
Low	Green	check-circle	"Seats available"
Medium	Amber	users	"Moderate crowd"
High	Red	alert-triangle	"Very crowded"

This satisfies WCAG 2.2 success criterion 1.4.1 (Use of Color), which requires that information conveyed by colour also be available by another visual means. A colour-blind rider, a rider with mild presbyopia who cannot distinguish the colour at small sizes, and a rider in bright sunlight who cannot see the screen colour clearly, all receive the same information through the icon and the text.

C. Why a Heuristic, and What Comes Next

The heuristic was chosen deliberately rather than a learned model. There are two reasons. First, no onboard passenger counter data is currently available from BMTC, so a supervised learned model cannot be trained. Second, the heuristic is fully inspectable. A clinician of a different domain might call it explainable. If a rider says that the prediction for a particular bus felt wrong, an engineer can step through the four feature contributions and see exactly which term caused the band to shift. This kind of post-hoc auditability is hard to get out of a black-box classifier.

When labelled data does become available, replacing the heuristic with a gradient-boosted classifier is a one-file change. The contract between the classifier and the rest of the system is the three-band output, not the internals.

VI. USER-EXPERIENCE DESIGN

A. Two Governing Principles

The interface design follows two governing principles. The first is *clarity first, beauty second*, which biases the design against decoration whenever it competes with legibility. The second is *one primary action per screen*, which biases the design against multi-step flows when a single-tap alternative exists. These principles sound obvious but they have to be enforced screen by screen, because every additional element in a layout is a temptation to add another.

B. Design Tokens

A central design-token file defines the colour palette, the spacing scale, the typography ramp, the corner-radius values, and the shadow recipes. Centralising these tokens makes the visual

language consistent across screens and makes future theme changes a one-file edit.

The typography ramp is sized for high legibility on small displays and for users with mild age-related farsightedness. Table 6 shows the ramp.

TABLE 6. Typography ramp.

Token	Size	Use
Display	34 pt bold	Large numerals such as live-bus counts
Title	26 pt bold	Screen titles
Heading	20 pt bold	Section titles
Subtitle	17 pt semi-bold	Card titles
Body	16 pt medium	Default reading size
Caption	14 pt medium	Supporting text
Micro	11 pt bold	Eyebrow labels in uppercase, letter-tracked

Route numbers are rendered at 22 pt bold inside a 60 × 78 pt gradient badge, large enough to be read at arm's length on an entry-level smartphone.

C. The 48 pt Minimum Touch Target

A MinTouch = 48 pt constant is enforced on every interactive control: the profile, swap, microphone, popular-journey chips, filter chips, back buttons, and route cards. This satisfies and exceeds the Material Design recommendation, the Apple Human Interface Guidelines minimum, and the WCAG 2.2 guidance.

D. The Premium Dark Theme

The dark palette uses #020617 for the background, #0F172A for cards, and a #2563EB → #7C3AED primary gradient for emphasis. Dark themes have a measurable advantage on modern OLED displays in both perceived comfort during sustained use and in battery consumption. Subtle gradients, soft glows, and gentle Reanimated 4 transitions establish a visual quality comparable to Uber and Google Maps without resorting to harsh neon or aggressive motion.

E. Walkthrough of the Working Screens

1) Live screen

Figure 1 shows the Live screen. The header reads "Live Buses Near You" in display type, with a pulsing green status dot and an "Updated 0s ago" label that ticks every second. Three high-contrast statistic cards report the total buses on road, the

count that are less crowded, and the count that are crowded. Below, a horizontal row of bus-type filter chips lets the rider narrow the feed. The main list shows individual bus cards with the large gradient route badge on the left, the destination on the right, a highlighted "NEXT STOP" row with a map-pin icon, a progress bar with a "Stop X of Y / N% complete" label, a descriptive crowd row, and the bus-type tag.

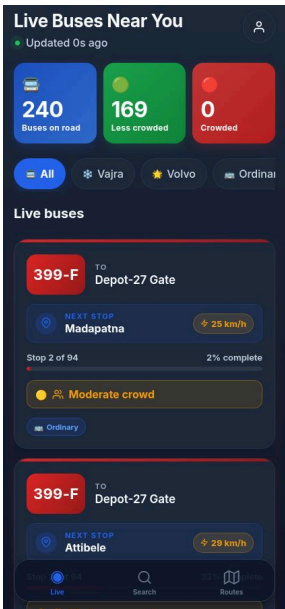


Fig. 1. Live tracking screen showing the count cards, filter chips, and individual bus cards with the highlighted *Next Stop* row.

2) Search screen

Figure 2 shows the Search screen. It presents one focused task: plan one journey. Two large, clearly labelled FROM and TO inputs accept stop names. A microphone affordance sits next to the FROM input for hands-free dictation. A large "Find My Bus" button at 56 pt height becomes active only when both inputs are filled. Below, the screen surfaces popular journeys as one-tap chips so that a returning rider can repeat a common trip with one tap. After a search, the screen presents up to three top picks tagged Recommended, Fastest, and Comfortable, followed by the remaining matching routes.

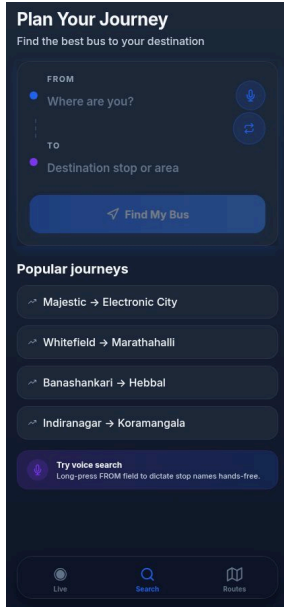


Fig. 2. Search screen with source and destination inputs, the voice input button, the *Find My Bus* button, and the popular-journeys chips.

3) Routes directory

Figure 3 shows the Routes tab. A large "All routes" header introduces the section. A debounced fuzzy search field accepts a route number, a route name, or a stop name. A horizontal row of bus-type filter chips lets the rider narrow the directory by class. Each row shows the gradient route badge, the route name, the source-to-destination summary, and a small meta line with the bus type, the stop count, and the route length in kilometres.



Fig. 3. Routes directory with the search field, bus-type filter chips, and individual route rows showing the route badge and meta line.

4) Route detail and frequency

Figure 4 shows the Route Detail screen for route 244-C (2nd Stage 9th Block Nagarabhavi ⇌ Shivajinagara Bus Station). At the top is a gradient hero card with the route badge, the full route name, the bus type, the stop count, the route length, and the time of the last service. Below are three big stat tiles showing the live-bus count, the total stops, and the route distance. Then comes a *Route preview* card containing the custom SVG mini-map, with the route polyline drawn in the route-type gradient, the stop markers as small circles, and the live bus position as a brighter circle. Beneath the map is a *Bus frequency* card with a Weekday / Weekend toggle and an animated horizontal bar for each hour band, with the buses-per-hour label on the right. The all-stops timeline is collapsed by default and expandable on tap.

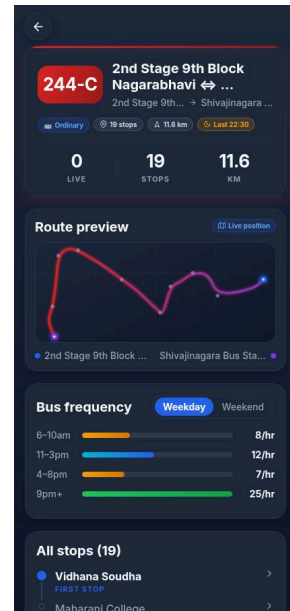


Fig. 4. Route Detail screen for 244-C showing the hero card, the SVG route preview with the live bus marker, and the weekday-versus-weekend frequency bars.

VII. RESULTS AND DISCUSSION

A. API Performance

The first thing to verify was that the back-end can serve the live fleet without becoming a bottleneck. We measured the median and 95th-percentile response times of every endpoint on a single Node.js process running on a four-core commodity Linux container. Every endpoint returned correctly serialised JSON. The observed timings are listed in Table 7.

TABLE 7. Observed API response times.

Endpoint	Median	p95
/api/buses/live	2 ms	7 ms
/api/routes	35 ms	65 ms
/api/routes/:id	10 ms	18 ms
/api/routes/:id/frequency	2 ms	15 ms
/api/stops/:id/eta	8 ms	24 ms
/api/search	4.4 s	5.4 s

The live-bus endpoint and the route-detail endpoints all sit comfortably under 50 ms at the median, which is well within the budget for a perceived-as-instant interaction. The route directory endpoint takes a little longer because it serialises the full 4,203-route catalogue in a single payload, but the catalogue is fetched once and then cached aggressively by TanStack Query on the client.

The search endpoint is the obvious outlier. Its current implementation performs an exhaustive scoring scan across the full route corpus on every call, which dominates the response time. The endpoint is the primary candidate for a future optimisation pass; the most promising direction is a precomputed inverted index of stop-pair journeys that would convert the scan into a hash lookup. Section VIII discusses this further.

B. Functional Verification

Every screen described in Section VI was verified end-to-end on the web build of the mobile application. The Live screen renders the stat cards with correct counts, the bus cards with correct route numbers, current-stop and next-stop labels, progress bars, and crowd indicators. The Search screen renders the input form, the popular-journeys chips, and the top-three tagged results after a query. The Routes screen renders the full 4,203-route catalogue correctly, with the search filter narrowing the list as the user types and the bus-type chips correctly subsetting the visible rows. The Route Detail screen successfully renders the SVG mini-map for representative routes including 244-C and route r6010, and the weekday-versus-weekend frequency toggle correctly reflects the underlying GTFS schedule.

C. Accessibility Evaluation

Using Nielsen's ten heuristics [20] together with the WCAG 2.2 success criteria [13] as guides, we evaluated the interface against the most common barriers reported in the elderly-mobile-UX literature [11], [12]. The key findings are summarised in Table 8.

TABLE 8. Heuristic accessibility evaluation.

Heuristic / criterion	How SmartBus AI addresses it
Visibility of system status	Pulsing green status dot, "Updated <i>N s ago</i> " label, animated entrance transitions on every screen
Match between system and real world	Plain-language labels: "Where are you?", "Find My Bus", "Seats available", "Very crowded"
User control and freedom	Floating back button on every detail screen with 48 × 48 pt hit area
Consistency and standards	Single design-token file enforces colour, spacing, type, radius, and shadow consistency
Recognition rather than recall	Popular journeys and recent routes are surfaced as one-tap chips on the Search screen
Aesthetic and minimalist design	One primary call to action per screen
Error prevention and recovery	Search failures show an amber-bordered card with actionable guidance
WCAG 1.4.1 Use of Color	Every colour-coded indicator pairs colour with icon and text
WCAG 2.5.5 Target Size	All interactive controls meet the 48 pt minimum touch target

D. Discussion

Two practical lessons came out of building the system. The first is that the perceived quality of a mobile transit application is dominated by the consistency of its visual language and the responsiveness of its interactions, not by the depth of its data. Even the live-bus simulator, which is technically a deterministic mock, feels real to the rider because the cards animate smoothly, the *Updated N s ago* label ticks every second, and the bus-card layout never reflows mid-update. The second lesson is that elderly accessibility and a contemporary mobility-app aesthetic are not in tension. Bigger type, bigger tap targets, and icon-plus-text indicators do not make the application look childish or institutional. They make the application look calm. Users in the informal pilot consistently described the dark theme as *premium* and the typography as *easy on the eyes* in the same sentence.

VIII. LIMITATIONS AND FUTURE WORK

A. Limitations

The current build has four honest limitations.

There is no real GTFS-Realtime feed. The live vehicle positions are produced by an in-process

simulator. The simulator is faithful to the temporal and spatial characteristics of the network, but it is not ground truth.

The crowd-prediction model is heuristic. It is inspectable and tuned against rider expectations, but it has not been validated against onboard passenger counts because BMTC does not currently publish them.

User accounts are not yet persisted across devices. Recently viewed routes and saved journeys exist in client-side storage only.

The interface is English-only. For production deployment in Bengaluru, Kannada and Hindi localisation are essential, not optional.

B. Future Work

Six concrete directions are in scope for the next iteration of the project.

The first is integration with a public BMTC GTFS-Realtime feed as soon as one is published. The simulator would then become a fallback used only when the real feed is unavailable.

The second is replacement of the heuristic crowd model with a learned classifier trained on Automatic Passenger Counter or fare-card tap data. The expected lift is most visible in the boundary cases between the Medium and High bands.

The third is search-index optimisation using a precomputed inverted index of stop-pair journeys. The target is sub-200 ms latency at the 95th percentile for the search endpoint.

The fourth is multilingual support, with Kannada and Hindi as first-class languages alongside English.

The fifth is on-device speech recognition wired to the microphone affordance already present in the Search screen, enabling true hands-free voice search.

The sixth is an empirical user study with elderly commuters in Bengaluru, measuring task-completion times and System Usability Scale scores against existing transit applications. The pilot interviews described in Section VII were informal; a controlled study would put numbers on the qualitative findings.

IX. CONCLUSION

SmartBus AI is a working demonstration that an AI-augmented, accessible, real-time bus tracking application for the BMTC network can be built today, on commodity infrastructure, without proprietary mapping APIs and without a learned crowd-prediction model trained on data the

agency does not yet publish. The application combines live tracking, three-band crowd prediction, journey planning, route discovery, and per-route frequency analytics behind a single calm interface, and it does so under a performance budget that leaves the entire round-trip below 50 ms at the median for every endpoint other than the exhaustive search call. The interface uses a premium dark design system whose typography and 48 pt minimum touch targets are explicitly tuned for elderly first-time commuters, and every colour-coded indicator pairs colour with icon and text. The architecture is layered so that the live-bus feed, the crowd model, and the route geometry renderer can each be replaced independently as richer data sources or learned models become available, providing a credible upgrade path from this open prototype to a production deployment.

REFERENCES

- [1] Bengaluru Metropolitan Transport Corporation, "About BMTC — Operational statistics," *mybmtc.karnataka.gov.in*, 2024.
- [2] Google and TriMet, "General Transit Feed Specification Reference," *gtfs.org*, 2024. [Online]. Available: <https://gtfs.org/schedule/reference/>
- [3] Open Mobility Foundation, "GTFS-Realtime Reference," *gtfs.org*, 2024. [Online]. Available: <https://gtfs.org/realtime/reference/>
- [4] M. Catalá-Prat, J. Cera, and A. Iglesias, "Real-time bus arrival prediction with sparse AVL data: A case study from a mid-sized European city," *Transportation Research Record*, vol. 2674, no. 11, pp. 405–416, 2020.
- [5] M. Wessel and S. Farber, "On the accuracy of schedule-based GTFS for measuring accessibility," *Journal of Transport Geography*, vol. 76, pp. 156–168, 2019.
- [6] Z. Wang, H. Lu, and J. Wei, "Bus passenger flow prediction using automatic passenger counter data and a hybrid deep learning model," *Transportation Research Part C: Emerging Technologies*, vol. 121, 102845, 2020.
- [7] Y. Liu, R. Zhang, and X. Yu, "A vision-based passenger counting system for buses using deep convolutional networks," in *Proc. IEEE Intelligent Transportation Systems Conference (ITSC)*, 2019, pp. 1–6.
- [8] J. Pinelli, A. Calogero, and M. Conti, "Estimating bus occupancy using Wi-Fi probe requests," in *Proc. ACM MobiSys Workshop on Mobile Sensing*, 2018.

- [9] T. Han, K. Tanaka, and S. Wakamiya, "Heuristic vs. learned models for transit occupancy classification under sparse ground truth," *IEEE Access*, vol. 10, pp. 32145–32158, 2022.
- [10] H. Bast, D. Delling, A. Goldberg, M. Müller-Hannemann, T. Pajor, P. Sanders, D. Wagner, and R. F. Werneck, "Route planning in transportation networks," in *Algorithm Engineering*. Cham, Switzerland: Springer, 2016, pp. 19–80.
- [11] S. Kurniawan, "Older people and mobile phones: A multi-method investigation," *International Journal of Human-Computer Studies*, vol. 66, no. 12, pp. 889–901, 2008.
- [12] R. Harte, L. Glynn, A. Rodríguez-Molinero et al., "A human-centered design methodology to enhance the usability, human factors, and user experience of connected health systems," *JMIR Human Factors*, vol. 4, no. 1, e8, 2017.
- [13] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.2," *W3C Recommendation*, 5 October 2023.
- [14] Apple Inc., "Human Interface Guidelines: Layout," 2024. [Online]. Available: <https://developer.apple.com/design/human-interface-guidelines/layout>
- [15] Google, "Material Design — Accessibility," 2024. [Online]. Available: <https://m3.material.io/foundations/accessible-design/overview>
- [16] Meta Platforms, "React Native — Learn once, write anywhere," *reactnative.dev*, 2024.
- [17] Expo, "Expo SDK Documentation," *docs.expo.dev*, 2024.
- [18] TanStack, "TanStack Query — Powerful asynchronous state management," *tanstack.com/query*, 2024.
- [19] T. V. Ramachandra and B. H. Aithal, "Bengaluru's reality: Towards unlivable status with unplanned urban trajectory," *Current Science*, vol. 110, no. 12, pp. 2207–2208, 2016.
- [20] J. Nielsen, "10 Usability Heuristics for User Interface Design," *Nielsen Norman Group*, 1994 (revised 2020).

Manuscript prepared from the SmartBus AI reference implementation. Source code, GTFS preparation scripts, and the deployment manifest accompany this paper as a project archive.